HABITFLOW

Project Documentation

A practical guide to the app, its technology, data handling, and interview discussion.

Project type	Purpose	Deployment
Mobile-friendly installable web app	Track habits and everyday wellbeing	GitHub Pages + Cloudflare push service

Quick summary

HabitFlow is a personal habit and routine tracker that runs in a web browser and can be installed on a supported phone or computer. People can record daily habits, plan time, log exercise and meals, track sleep, set goals, review trends, and schedule reminders. The main app is static and stores personal tracking records locally in the browser.

This guide describes the current implementation in the project source. It is not a claim that every feature is production-grade or that personal records automatically synchronize between devices.

1. What the app does

Main features

Area	What it provides
Overview and My Day	A daily summary, tasks, planned time blocks, and progress for the selected day.
Habit Tracker	Create and mark recurring habits; view completion and streak-style progress.
Health and routine logs	Diet/meal entries, workouts and running, sleep duration and quality, mood and energy.
Study and focus	Tasks, estimated/actual time, focus timer, and saved focus sessions.
Goals and insights	Targets, daily scoring, reports, weekly summaries, and historical analytics.
Settings and backups	Adjust score weights and targets, choose 12/24-hour display, export JSON/CSV, restore a JSON backup, or clear local records.
Reminders and install	Set daily reminder times, enable browser notifications, and install HabitFlow to a device home screen where supported.

Typical user flow

Open the app, review the day, mark completed habits, add a task or time block, record optional health/activity data, and check reports later. Settings includes backup controls so a user can export records before changing devices or clearing browser data.

2. Languages and technologies

Core languages

Language	Use in this project
HTML5	Page structure, forms, navigation, and app content. The primary interface is in index.html.
CSS3	Responsive layout, colors, spacing, and phone-friendly presentation; styles are embedded in the main HTML file.
JavaScript (vanilla)	App views, form behavior, scoring, tracking, local storage, exports, reminders, and installation flow. No front-end framework is used.
JSON	Configuration and data formats, including the PWA manifest and JSON backup files.
SQL	Cloudflare D1 table schema for device push subscriptions and reminder schedules.

Platform and browser APIs

Progressive Web App (PWA): manifest.webmanifest describes the installable app; sw.js caches app files for offline loading and handles background push notifications.

Browser APIs: localStorage stores app records; Service Worker, Cache API, Notifications, Push API, and the install prompt support offline use, reminders, and installation. Support differs by browser and operating system.

Hosting: GitHub Pages serves the static website over HTTPS.

Background reminders: a Cloudflare Worker exposes the push API; Cloudflare D1 keeps device subscription and reminder schedule metadata. Web Push uses VAPID keys. The main app records are not sent to this reminder service.

High-level architecture

Phone / browser HTML + CSS + JavaScript Local app records + PWA cache	Website files HTTPS	**GitHub Pages** Static app hosting
Reminder schedule + device push subscription	Reminder API	**Cloudflare Worker + D1** Background notification delivery

3. Data, privacy, and practical limits

Where data is stored

Daily habits, tasks, meals, workouts, sleep, goals, and most settings are saved in localStorage in the current browser profile on the current device. That means data does not automatically appear on another phone or in another browser. Export a JSON backup from Settings & Data and restore it on the other device if you want to move a copy.

The reminder service is separate. When background push is enabled, the app sends a device push subscription, a device identifier/secret, time zone, and reminder names/times to the Cloudflare service. It does not upload diary entries, habit history, health logs, or profile data for those reminders.

Important limits

• There is no user login or account-based sync in this version.
• Clearing browser site data can erase local records; export backups regularly.
• Install and notification support varies by browser/device. Users must grant notification permission on each device.
• Background notification arrival depends on push services, internet access, OS permissions, and battery settings.
• The app is a personal organizer, not a medical device or professional health advice.

Interview preparation: sample questions and answers

What is HabitFlow?

It is a mobile-friendly personal habit and routine tracking web app. It helps users record daily habits, schedule tasks, track activity and sleep, set goals, and review progress.

Which languages did you use?

HTML5 for structure, CSS for responsive styling, JavaScript for application logic, JSON for configuration and backups, and SQL for the Cloudflare D1 reminder schema.

Did you use React or another framework?

No. The current front end uses vanilla JavaScript and a single-page HTML interface, which keeps the static app lightweight and deployable on GitHub Pages.

How is the app data saved?

The main records are serialized as JSON in browser localStorage. The app also offers JSON backup and restore, plus CSV export for selected records.

How does installation work?

It is a Progressive Web App. A web manifest describes the app and a service worker supports caching. Supported browsers offer an install prompt; on other devices the user can use the browser menu to add it to the home screen.

How do reminders work when the app is closed?

The browser subscribes the device to Web Push. The Cloudflare Worker and D1 store reminder metadata and schedule delivery; the service worker displays the push notification. Delivery depends on device and browser support.

Does the app sync a user’s logs across phones?

Not automatically. The current logs remain in each browser’s local storage. Users can export a JSON backup and restore it on another device.

Why use a service worker?

It enables the PWA install experience, caches app files for offline loading, and receives push events to display notifications while the page is not open.

How is the app hosted?

The front end is published as a static site through GitHub Pages over HTTPS. Background push is handled separately by Cloudflare Workers and D1.

What would you improve next?

I would add automated UI and data-migration tests, improve backup validation, provide optional secure account sync, and monitor notification delivery and accessibility across devices.

Interview tip: be transparent about what is implemented today. In particular, explain local-only tracking data separately from the Cloudflare reminder service; do not describe it as full account sync.